Project 2
Name: Uri Naor
Date: April 12, 2026

1. **Summary**
    a. **Describe your unit testing approach for each of the three features.**
        i. **To what extent was your approach aligned to the software requirements? Support your claims with specific evidence.**

I believe my overall tests were effective and well aligned to the requirements because I tried to stick to the requirements as thoroughly as possible. For example, the contact, task and appointment classes had the requirements to have an ID field that is less than 10 characters and not null.

I tested both of these requirements like so:

```
@Test
 void testConstructorInvalidId() {
    // null id
    assertThrows(IllegalArgumentException.class, () -> {
        new Contact(null, "Uri", "Naor", "0534567890", "Awesome St 47");
    });
    // id is too long
    assertThrows(IllegalArgumentException.class, () -> {
        new Contact("12315518901", "Uri", "Naor", "0534567890", "Awesome St 47");
    });
 }
```

Where the arguments for the constructor were valid except for the ID.

And I followed this approach, trying to closely align with the requirements, for all the tests that I wrote.

ii. Defend the overall quality of your JUnit tests. In other words, how do you know your JUnit tests were effective based on the coverage percentage?

Overall I'm confident in the quality and effectiveness of my JUnit tests. As stated, I stuck to the requirements very thoroughly, and my tests achieved a high coverage percentage. A 98% overall line coverage and a 92% overall branch coverage when taking into consideration all three assignments.

Coverage | All in ProjectOne

Element	Class, %	Method, %	Line, %	Branch, %
all	100% (12/12)	100% (74/74)	98% (250/254)	92% (63/68)
Appointment	100% (1/1)	100% (6/6)	100% (15/15)	100% (12/12)
AppointmentService	100% (1/1)	100% (4/4)	100% (8/8)	100% (2/2)
AppointmentServiceTest	100% (1/1)	100% (3/3)	100% (16/16)	100% (0/0)
AppointmentTest	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
Contact	100% (1/1)	100% (10/10)	100% (28/28)	96% (25/26)
ContactService	100% (1/1)	100% (8/8)	87% (21/24)	70% (7/10)
ContactServiceTest	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
ContactTest	100% (1/1)	100% (8/8)	100% (36/36)	100% (0/0)
Task	100% (1/1)	100% (6/6)	100% (15/15)	100% (12/12)
TaskService	100% (1/1)	100% (6/6)	93% (15/16)	83% (5/6)
TaskServiceTest	100% (1/1)	100% (5/5)	100% (26/26)	100% (0/0)
TaskTest	100% (1/1)	100% (6/6)	100% (20/20)	100% (0/0)

These high coverage percentages indicate that my tests thoroughly evaluated the code. My tests successfully executed almost all of the conditional branches and error handling logic in my constructors and functions, proving that the application handles invalid arguments and

operations as expected.

b. **Describe your experience writing the JUnit tests.**

i. **How did you ensure that your code was technically sound? Cite specific lines of code from your tests to illustrate.**

I ensured that my code was technically sound by utilizing a defensive programming approach, meaning that I validated inputs before allowing them to influence the state of the object. For example inside the constructor, if the passed ID was invalid, i'd throw an exception immediately. Like so:

```
    if (contactId == null || contactId.length() > 10) {
        throw new IllegalArgumentException("contactId can't be null or longer than 10 characters");
    }
```

And only then I updated the actual contact ID field after I was sure it was valid:

```
    this.contactId = contactId;
```

I then tested the constructor function, that this exception is indeed thrown when an illegal argument is passed:

```
// null id
Assertions.assertThrows(IllegalArgumentException.class, () -> {
   new Contact(null, "Uri", "Naor", "0534567890", "Awesome St 47");
```

```
});
// id is too long
Assertions.assertThrows(IllegalArgumentException.class, () -> {
    new Contact("12315518901", "Uri", "Naor", "0534567890", "Awesome St 47");
        });
```

ii. How did you ensure that your code was efficient? Cite specific lines of code from your tests to illustrate.

I ensured my code was efficient by choosing a data structure that fits well for the job. I went with a HashMap because it offers a highly optimized time efficiency for lookups, insertions and deletions.

A good example of this efficiency is the delete contact test:

```
  @Test
   void deleteContact() {
       ContactService contacts = new ContactService();
       contacts.addContact("1", "Uri", "Naor", "0534567890", "Awesome St 47");
       contacts.deleteContact("1");

       // readding deleted
       Assertions.assertDoesNotThrow(() -> contacts.addContact("1", "Uri", "Naor",
"0534567890", "Awesome St 47"));
   }
```

Notice I am able to instantly delete the contact from the data structure by passing the unique ID. Thus skipping the need to manually iterate inefficiently through a list.

2. Reflection

a. Testing Techniques

i. What were the software testing techniques that you employed in this project? Describe their characteristics using specific details.

For each of the milestones I employed Unit Testing. Unit Testing is a testing technique in which we test smallest parts of an application, such as individual methods, making sure they behave as expected, and produce the expected output for the expected input, including edge cases.

To be specific, I tested the individual methods of the Contact and ContactService classes, of the Task and TaskService, and recently of the Appointment and AppointmentService. In all of the unit tests that I wrote I made sure to check both valid and invalid inputs to assert that the methods throw exceptions for invalid inputs, and behave as expected when the input they receive is valid.

For example, there was a requirement for Appointment class, that the description can't be null or longer than 50 characters.

I tested the setDescription method by first passing valid input - a description that isn't null but is shorter than 50 characters "super important meeting with the president", I used the set method to change it to another valid description "even more important meeting with justice league", and asserted that the description was successfully changed with the getter method. I

later checked that the method successfully throws exceptions for invalid input, such as a null description or a description longer than 50 characters.

```
  @Test
   void setDescription() {
       Date oneMinFromNow = new Date(System.currentTimeMillis() + 60000);
       Appointment appointment = new Appointment("1337", oneMinFromNow, "super
important meeting with the president");

       assertEquals("super important meeting with the president",
appointment.getDescription());
       appointment.setDescription("even more important meeting with justice league");
       assertEquals("even more important meeting with justice league",
appointment.getDescription());

       assertThrows(IllegalArgumentException.class, () ->
appointment.setDescription(null));
       assertThrows(IllegalArgumentException.class, () -> appointment.setDescription("This
is a long and detailed description that is definitely longer than 50 chars."));
   }
```

ii. What are the other software testing techniques that you did not use for this project? Describe their characteristics using specific details.

Since I used Unit Testing exclusively, I did not implement Integration Testing. Integration Testing is a testing technique in which we integrate our module inside a system composed of

multiple modules, and we test how well the different components interact with each other to make sure everything is behaving as expected and that there are no pieces that suddenly broke as a result of the integration.

I also didn't do Black Box Testing. Black Box Testing is a technique in which we write tests for different inputs, both regular and edge cases based on what we expect the components to support without directly viewing the implementation. This method encourages us to think like the user rather than like the developer. Because I wrote the code right before writing the tests I did White Box Testing, which means I wrote the tests while looking at, and keeping in mind the actual implementation.

iii. For each of the techniques you discussed, explain the practical uses and implications for different software development projects and situations.

Unit Testing is practically used as a first line of defense. Its main implication is that it provides quick feedback that allow us to catch logic bugs immediately without having to run the entire application.

Integration Testing is extremely important to make sure that the system behaves as expected as a whole. If we skip integration testing we might find out that things break apart unexpectedly.

For example if the Appointment Service class was needed to connect with a database, it would be a good idea to make sure that the code can successfully write and retrieve data from it. If we wouldn't test the integration we might find out later that Appointment Service

doesn't behave properly, but it would be harder to find that out, because we probably have already introduced multiple modules by that point, and the entire system is more complex and has more moving parts that could potentially lead to the failure. The milestone code was completely self contained so there was no integration to test in our case.

Black Box Testing is important because it provides a mental shift. It makes us more creative and think like a user. As developers we might not think of the same use cases. The advantage of White Box Testing though is that we might be more aware of potential downfalls. So there isn't a distinct advantage of Black Box Testing over White Box. Simply different advantages and disadvantages.

b. Mindset

i. Assess the mindset that you adopted working on this project. In acting as a software tester, to what extent did you employ caution? Why was it important to appreciate the complexity and interrelationships of the code you were testing? Provide specific examples to illustrate your claims.

As both the developer and the tester of this project, I definitely had to employ a good deal of caution. Even though the applications I wrote were mostly simple, such as using built-in data structures to store the objects that we define ourselves, rather than connecting to a complex database for example, I still felt that the code has important interrelationships which were important to keep in mind. For example, all "Service" classes like the TaskService, ContactService and AppointmentService classes must first verify if a unique ID already exists in the HashMap before adding another object, such as another Appointment to the map.

If I didn't consider that complexity, I might have written tests that only check the successful updates and completely missed edge cases such as when a user tries to modify a non-existent ID.

ii. **Assess the ways you tried to limit bias in your review of the code. On the software developer side, can you imagine that bias would be a concern if you were responsible for testing your own code? Provide specific examples to illustrate your claims.**

I believe that limiting your bias is one of the hardest parts of testing your own code. Because I wrote the implementation right before writing the tests, I was exclusively practicing White Box testing. While White Box testing can help me focus on the specific "tricky" parts of my implementation, the inherent danger here was that I won't think like a real unpredictable user. I already knew the happy path that I expect my code to take, which made it easy to unintentionally ignore the paths that break my code, which is the exact goal of testing. To limit my own bias, I tried to view my code through a lens of a QA engineer who is actively trying to think of edge cases to break my code. I focused on finding ways to intentionally break my own code with bad input, as well as making sure the "happy path" is working as intended as well. I did this by doing things like passing a date in the past for an appointment, or sending a description which is over 50 characters to see if an exception is properly thrown, while also passing valid dates and descriptions to make sure they work as intended.

iii. **Finally, evaluate the importance of being disciplined in your commitment to quality as a software engineering professional. Why is it important not to cut corners when it comes to writing or testing code? How do you plan to avoid technical debt as a practitioner in the field? Provide specific examples to illustrate your claims.**

I actually already work part time as a software developer, and I can confidently claim that it is extremely easy to skip proper testing.

In real life, there are deadlines, new features are expected and needed to be shipped and testing often feels like a chore. It is easy to feel like the code we wrote is perfect before testing it. However testing is extremely important, especially if we want to leave a good impression on our employer and ship functional, reliable code.

There's not much worse than pushing broken code into production and then being asked to see the tests you didn't write during code review. But there is worse, if the code wasn't

caught during the code review and is actually pushed into production, it can become much harder to find the bug, and it can cause bigger issues down the line.

While it might be annoying and tedious, in a professional setting, the discipline to write tests is critical. While cutting corners can be tempting, it creates technical debt that we should remember will always cost us more later on. Debugging would become more complex and time consuming, and the damage would be greater.

By taking the hard route today, we set ourselves to take the easy route later, and enjoy the peace of mind that comes from knowing our code is reliable. Well mostly, as there is no such thing as perfectly tested code.